REPORT

Scalability Gap Analysis

COE 454 inventory and sales app for one stationery shop. Scores are architectural fit, not load-test measurements. Source: Logical883/regimens codebase, assessed 10 Sep 2026.

Verdict: sized for one shop, not a chain

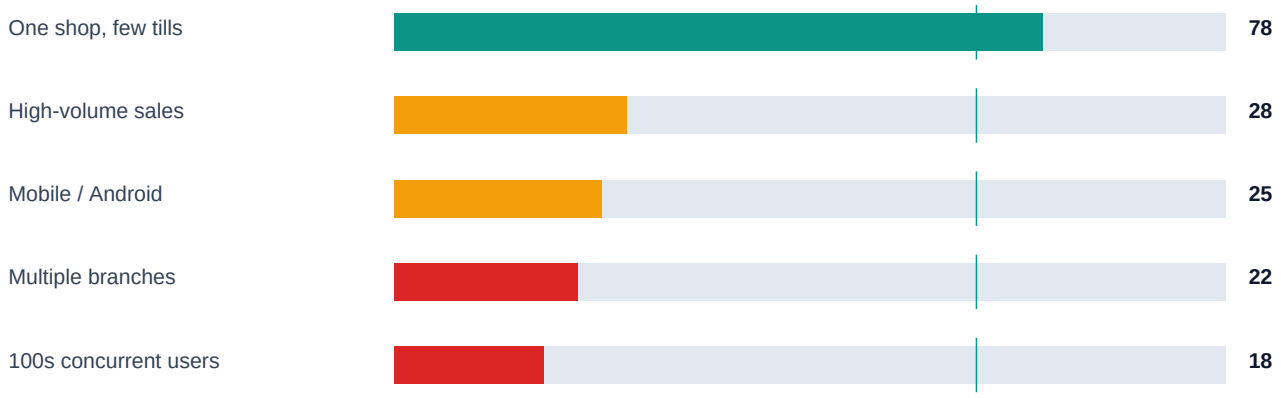
The stack is a modular monolith: browser -> Vercel React SPA -> same-origin /api proxy -> one Express process on Render -> Neon/Postgres (or in-memory mock). That is enough for a few counters. It is not a multi-branch, high-concurrency, or native-mobile platform.

1	300	60 s	12 h
Express process	API req / 15 min / IP	Proxy + client timeout	JWT lifetime (no revoke)

1. Fit by growth scenario

Architectural fit (0-100) by target load. A score of 70 or above means the current design can ship as-is for a demo shop.

Architectural fit (0-100). Dashed target = 70 (demo shop).



Scenario	Fit	What blocks scale
One shop, few terminals	Good (with Neon)	Render sleep/cold start; filesystem backups on ephemeral disk; KeepAlive needs a tab or uptime ping.
High-volume sales	Weak	Sync batch max 100 sales / 100 products; JSON body 256kb; FIFO row locks; unbounded list/profit queries.
Mobile / Android	Web only	Vite SPA + Dexie in the browser. No Capacitor, PWA worker, or Android package in the repo.
Multiple branches	Not modeled	No store/branch/tenant id. One global users + inventory schema. Last-write-wins assumes one shop.

Hundreds of concurrent users	Poor	Single Node process; in-memory rate limits; default pg pool (~10); no Redis/queue/CDN; hot SKU lock contention.

2. What already holds for class / one shop

Durable mode (USE_MOCK_DB=false + Neon) uses transactional sales, FIFO cost layers with FOR UPDATE, and stamped unit_cost / unit_price on lines. Owner vs assistant roles, parameterized SQL, helmet/CORS, and a Dexie offline sale queue match a small counter with flaky internet. Split hosting (Vercel UI, Render API, Neon DB) is the intended deploy path.

3. Hard ceilings in the current design

Single process (backend/src/index.js)

app.listen only. No cluster, workers, or replica set. Mock state and express-rate-limit live in that process memory, so they cannot be shared even if you added instances.

Free-tier hop (frontend/api/proxy.js)

Vercel function maxDuration 60s; axios timeout 60s; documented Render cold start 30-60s. KeepAlive pings every 30s with a 10s health timeout. First sale after sleep can still miss the window.

Data and sync

Product sync is last-write-wins: if the server row is newer, the offline edit is dropped (backend/src/routes/sync.js). Sales flush in one POST. listSales and profit reports load matching rows then N+1 item queries (backend/src/data/real.js) with no pagination. Pool is new Pool() with no max / idle settings (backend/src/db.js).

Mock mode (USE_MOCK_DB=true) resets on every sleep or restart - fine for a marking demo, not for a live shop. Backups write JSON under database/backups/ on the API host disk, which Render does not treat as durable storage.

4. Limits taken from code

Limit	Value	Where
JSON body	256 kb	backend/src/index.js
Global rate limit	300 / 15 min	security.js RATE_LIMIT_MAX
Login / forgot limit	20 / 15 min	security.js LOGIN_RATE_LIMIT_MAX
JWT expiry	12 h default	auth.js JWT_EXPIRES_IN
JWT secret floor	32+ chars	security.js, render-start.js
Axios timeout	60 000 ms	frontend/src/api/client.js
KeepAlive interval	30 000 ms	KeepAlive.jsx
KeepAlive health timeout	10 000 ms	KeepAlive.jsx
Vercel proxy maxDuration	60 s	frontend/api/proxy.js

Sync batch	100 sales, 100 products	backend/src/routes/sync.js
Sale lines / qty	200 lines, qty <= 100 000	backend/src/routes/sales.js
Render cold start (doc)	~30-60 s	docs/DEPLOY.md

Values are configured defaults, not measured production traffic.

5. If the shop grows - order of work

Priority	Change	Why it matters
P0	Paid always-on API + Neon (no mock, no sleep)	Removes the 30-60s cold-start cliff that currently looks like API unreachable.
P1	Paginate sales/reports; set pg pool max	Stops unbounded reads and connection stampede before adding machines.
P1	Idempotent sync keys; raise or chunk batches	Offline retries must not double-post sales under load.
P2	Shared rate-limit store; JWT refresh/revoke	Needed only if you run more than one API instance.
P3	branch_id schema; native/PWA client	New product, not a config tweak - current model is one shop.

Course vs production

For COE 454, the gap that matters in a demo is hosting (sleep, suspended Render, proxy), not Kubernetes. Treat Redis, multi-region, and Android as future work, not missing homework, unless the brief asks for them.